# Umuriro Response: Technical Report

Electricity fault reporting and technician response for Ndera Sector, Kigali. Repository: https://github.com/Larissanoella-05/Capstone-project · Live app: https://capstone-project-six-inky.vercel.app

## 1. Scope alignment

When the power fails, people phone the utility and describe where they live, and the technician often calls back to find the house. Umuriro Response keeps one shared, located record: the customer reports, the coordinator assigns, the technician updates the status, the customer follows. Full detail: SCOPE_ALIGNMENT.md.

**Table 1. Planned features and what was built**

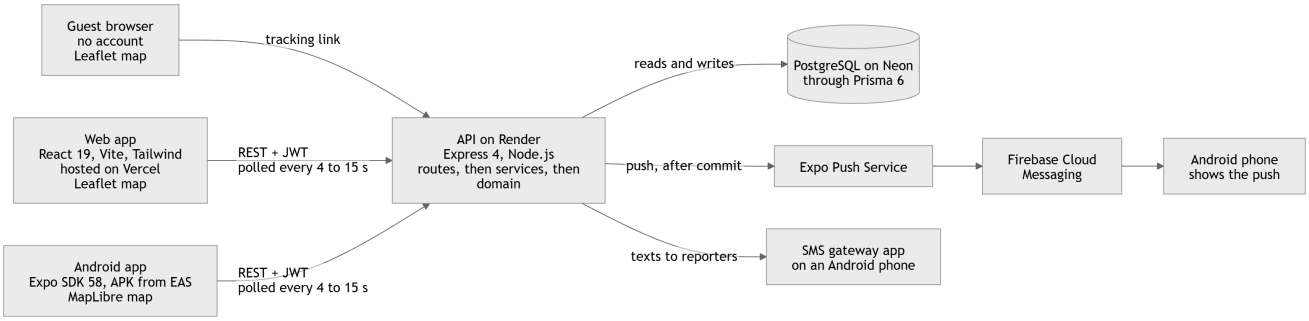
| Planned feature | Status | Evidence |
|---|---|---|
| Sign-in for three roles (FR1) | Completed; customers register by email or phone | `middleware/auth.js` |
| Report linked to a service location | Completed | `faultService.createFault` |
| Report without an account | Completed (added after feedback) | `POST /api/public/faults` |
| Coordinator dashboard and queue | Completed | `pages/coordinator/Dashboard.jsx` |
| Ranked technician assignment | Completed | `TechnicianRanker` |
| Time-stamped status workflow | Completed | `FaultStatusMachine` |
| Customer and guest tracking | Completed | `pages/customer/Tracking.jsx`, `pages/public/` |
| Live technician position | Real GPS on Android, simulated on web | `ActiveJobTracker` |
| Arrival distance and analytics | Completed | `statsService` |
| SMS to the reporter | Partial: real phone, local run; deployed app logs only | `smsService.js` |
| Administrator area (FR13) | Not built | Accounts come from the seed script |

**Where the build differs from the proposal**

- *Report without an account:* the supervisor said nobody should log in during an emergency.
- *Technology:* JavaScript checked with JSDoc and `tsc` (not TypeScript), plain PostgreSQL (not PostGIS), polling every 4 to 15 s (not Socket.IO), Leaflet with OpenStreetMap (not Google Maps). This removed a paid map key and a socket server from a free hosting plan.
- *Android app:* added because a browser tab cannot keep sending a technician's position with the screen locked.
- *SMS:* through an Android phone with an open-source gateway, since a paid account was not an option.
- *Workflow: Accepted* dropped (assignment is the commitment), *Working* added to separate travel from time on site.
- *Evaluation:* the proposed pilot (about 20 customers, 5 technicians, 3 coordinators) has not run. Results come from automated tests, a load test and a one-person walk through the live workflow with demo data. No response-time claim is made about real users.

## 2. Technical functionality

**Architecture.** One Express API and one PostgreSQL database serve the React web app and the Expo Android app, hosted on Vercel, Render and Neon (Figure 1).




**Request flow.** `routes/` (HTTP) → `middleware/` (token, role, Zod validation) → `services/` (rules, database) → `domain/` (pure rules). A report is saved with its first history row and a notification in one transaction. Each later status change saves a time, history row and notification together. At *Arrived* the distance from the technician's last position is stored. Texts and push go out after the transaction and never fail a request.

**Guest reports.** No customer record. A 24-byte random tracking token gives a link that shows progress but never the phone or meter number and expires 7 days after resolution. Limit: 3 reports per phone per hour; a hidden form field catches simple bots.

**Algorithms**

- *Technician ranking:* availability, then distance (haversine), then workload, then name. Deterministic, O(n log n).
- *Optimistic concurrency:* `guardedUpdate` runs `UPDATE … WHERE` on the status that was read; a stale update matches no row and returns 409, with no database locks.
- *Work queue:* unassigned first, oldest first; late after 15 minutes (`lib/workQueue.js`, pure).
- *Photo shrinking:* photos over 1 MB are redrawn on a canvas as lower-quality JPEG until they fit.
- *Location throttling:* a phone position is sent after 20 s and 30 m of movement.
- *Paging and SQL averages:* the list is paged; averages use `AVG … FILTER` and `percentile_cont`.

**Error handling.** Zod validates every body and one handler returns `{ error: { message, details } }`: wrong password 401, non-Rwandan phone 400, another customer's fault 403, skipped stage 409, fourth guest report in an hour 429, duplicate phone 409, wrong tracking token 404.

## 3. Testing and results

Details and commands: TESTING.md.

**Table 2. Approaches**

| Approach | Why | Result |
|---|---|---|
| Unit | Workflow, ranking, validation break quietly | 534 pass (backend 222, frontend 148, mobile 164) |
| Integration | Roles hand work over through one database | 104 pass, 27 suites, separate test database |
| Load | Every open screen polls the API | Table 3 |
| Code review of both apps | Finds what tests miss | 8 problems fixed |
| Manual device and usability | Maps, GPS, camera, SMS need a real phone | Compatibility and Lighthouse below; usability not run |

**What testing changed.** Five quick "On the way" taps all succeeded with duplicate history, and two coordinators could assign one fault; optimistic concurrency fixed both, and the tests fail without it. The load test showed a 3 MB fault list at 2,000 faults (now paged) and slow analytics (averages moved into SQL).

**Table 3. Load test, 2,028 faults, local laptop** (single runs, about 30% variation)

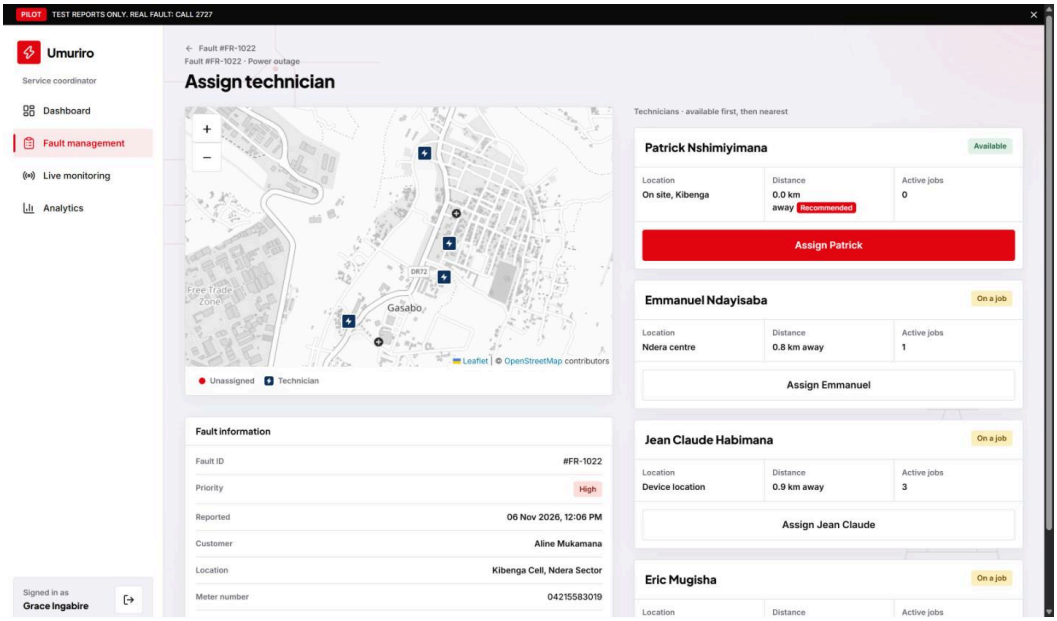
| Request | Before | After |
|---|---|---|
| Fault list | 3,105 KB, p50 3,743 ms | 28 KB, p50 72 ms |
| `GET /stats/analytics` | p50 319 ms | p50 67 ms |
| `GET /stats/overview` | p50 106 ms | p50 36 ms |

On the live Render API (1,600 requests, 0 errors) dashboard endpoints answered in 345 to 616 ms at the median, under 1 s at p95. One fault (717 KB of photos) took p50 869 ms. The first request after the free tier slept took 24.4 s; the next took 0.6 s.

**Compatibility.** Every screenshot and live check used desktop Chrome (1440 × 840). Firefox, Safari and Edge were not tested. On a TECNO KG5j (Android 11) the installed app signed in as customer and technician, created an account, reported a fault and moved a job through every stage; real texts reached an iPhone in a local run. Push worked with the app open only.

**Lighthouse, live site, mobile profile** (Lighthouse 12.8.2, Chrome 155, 10 Nov 2026, landing page): performance **64**, accessibility **98**, best practices **100**, SEO **92**. First contentful paint 3.6 s, largest contentful paint 5.6 s, total blocking time 240 ms, layout shift 0. Performance is the weak score; it was not tuned.

**Usability.** A 5-task usability test and an acceptance survey (SUS) were prepared but not run, so there is no user-satisfaction result. It is the main piece of future work.




## 4. Code quality

- **Structure:** `backend/`, `frontend/`, `mobile/`, `docs/`. The backend flows one way from routes to services to domain; no file mixes HTTP, business rules and queries.
- **Encapsulation:** `FaultStatusMachine` holds every workflow rule with no database code, so it is tested alone.
- **Polymorphism:** `SmsProvider` is a base class with a logging provider and `SmsGateProvider`; notice code calls `send()` and does not know which it has.
- **Errors:** `HttpError` helpers (`badRequest`, `notFound`, `conflict`) all pass through one handler.
- **Honest limit:** the service layer uses functions, not classes; we did not add classes for show.
- **Consistency:** ESLint with zero warnings, Prettier and a backend type-check run on every commit and in GitHub Actions, where the `ci-ok` job must pass before a merge. All web HTTP goes through `api.js`.
- **Process:** 190 commits from 26 September to 10 November 2026, Conventional Commits, merged by pull request. Database changes are committed Prisma migrations, never edited after being applied.
- **Known limits:** photos as data URLs, simulated web positions, polling, SMS not on the deployed app, no administrator area, type-checking on the backend only. Each is a documented prototype decision.

## 5. Conclusion

The report-to-resolution flow works for three roles on web and Android, and for people with no account. It is guarded against double taps and wrong-role access and covered by 638 automated tests. Load testing found two slow endpoints, both fixed and re-measured. Not built: the administrator area. Still to do: browser coverage beyond Chrome and a usability test with real users.